

Domanda 1.

Si consideri la seguente base di dati (chiamata “Incarichi”) per la gestione degli incarichi da Primo Ministro della Repubblica Italiana:

PRIMOMINISTRO(Cognome, Nome, PartitoPolitico*, AnnoNascita)

LEGISLATURA(Numero, DataInizio, DataFine*)

INCARICO(CognomePM, NomePM, NumeroL, DataPreincarico, DataGiuramento*, Durata*)

La LEGISLATURA, della durata tipica di cinque anni, è caratterizzata da un numero, da una data di inizio e una data di fine legislatura.

Nella relazione INCARICO, CognomePM e NomePM costituiscono una chiave esterna su PRIMOMINISTRO, mentre NumeroL è in vincolo di integrità referenziale con LEGISLATURA. DataPreincarico è la data in cui il Primo Ministro riceve l’incarico (eventualmente esplorativo) da parte del Presidente della Repubblica. Se il preincarico (esplorativo) va a buon fine (cioè, il Primo Ministro individua una squadra di ministri e una maggioranza in parlamento), DataGiuramento contiene la data di effettiva formazione (corrispondente al giorno del giuramento) del governo presieduto dal Primo Ministro. Durata contiene invece la eventuale durata del mandato. Gli attributi asteriscati possono essere nulli. Quando PartitoPolitico è nullo, il Primo Ministro è un tecnico (ovvero, non appartenente ad alcun partito politico).

Con riferimento alla base dati “Incarichi” si richiede di rispondere alla seguente domanda in SQL:

Individuare la legislatura con il maggior numero di preincarichi a tecnici non andati a buon fine.

Soluzione 1

```
WITH num_incarichi AS (  
    SELECT NumeroL, COUNT(*) AS Num  
    FROM incarico JOIN primoministro ON (CognomePM=Cognome AND NomePM=Nome)  
    WHERE PartitoPolitico IS NULL AND DataGiuramento IS NULL  
    GROUP BY NumeroL)  
SELECT NumeroL  
FROM num_incarichi  
WHERE Num >= ALL (SELECT Num FROM num_incarichi);
```

Altre soluzioni sono possibili.

Domanda 1.

Si consideri la seguente base di dati (chiamata “Incarichi”) per la gestione degli incarichi da Primo Ministro della Repubblica Italiana:

POLITICO(Cognome, Nome, DataNascita)

ADESIONE(Cognome, Nome, NomePartito, DataAdesione)

PARTITO(NomePartito, Area*, DataFondazione)

PRIMOMINISTRO(Cognome, Nome, DataIncarico, Durata*)

Nella relazione PRIMOMINISTRO, *Cognome* e *Nome* costituiscono una chiave esterna su POLITICO. *DataIncarico* è la data in cui il politico avente *Cognome* e *Nome* riceve l’incarico (eventualmente esplorativo) da parte del Presidente della Repubblica. *Durata* contiene invece la eventuale durata (in giorni) del mandato ed è null se l’incarico non è andato a buon fine (cioè, se il politico non ha individuato una squadra di ministri e una maggioranza in parlamento).

In ADESIONE l’attributo *NomePartito* è una chiave esterna su PARTITO. *DataAdesione* indica invece la data in cui il politico *Cognome*, *Nome* ha aderito al partito politico indicato in *NomePartito*. Un politico **non può** appartenere a più partiti contemporaneamente.

In PARTITO l’attributo *Area* indica l’area di appartenenza del partito (destra, centro-destra, centro, centro-sinistra, sinistra). Se è null, il partito non fa riferimento a nessuna area politica tradizionale.

Si suppone che un Primo Ministro non possa cambiare partito politico durante il suo incarico.

Con riferimento alla base dati “Incarichi” si richiede di esprimere in SQL la seguente richiesta:

Individuare l’area politica tradizionale che ha avuto il maggior numero di politici incaricati che hanno effettivamente governato, cioè i cui incarichi sono andati a buon fine.

Soluzione 1.

Una possibile soluzione è la seguente:

```
with PrimoMinSuaArea as (  
    select distinct(p.Cognome, p.Nome, p.Area)  
    from PRIMOMINISTRO pm join POLITICO p using (Cognome, Nome)  
    join ADESIONE a using (Cognome, Nome)  
    join PARTITO pp using (NomePartito)  
    where (pm.Durata is not null)  
    and (pp.Area is not null)  
    and (pm.DataIncarico>=a.DataAdesione)  
    and NOT EXISTS (select *  
                     from ADESIONE a2  
                     where (a2.DataAdesione>a.DataAdesione)  
                           and (a2.DataAdesione<=pm.DataIncarico))  
),  
with AreaQuantipM as (  
    select Area, count(*) as NroIncSucc  
    from PrimoMinSuaArea  
    group by Area  
),  
select Area, NroIncSucc  
from AreaQuantipM  
where NroIncSucc = (select max(NroIncSucc)  
                   from AreaQuantipM);
```

Domanda 1.

Considerare la base dati “Cinguettii” contenente dati relativi ad un sito di microblogging con il seguente schema

UTENTE(IDUtente, Nome, Città)

SEGUACE(IDUtente1, IDUtente2, DataS)

CINGUETTIO(IDCinguettio, IDUtente, Testo, DataC, Ricinguettio)

Ogni tupla di SEGUACE rappresenta la seguente informazione: l’utente identificato da IDUtente1 inizia a seguire l’utente identificato da IDUtente2 nella data definita da DataS.

Ogni tupla di CINGUETTIO rappresenta un “micropost”, pubblicato nella data definita da DataC, dall’utente identificato da IDUtente. Se un micropost A cita un altro micropost B, l’attributo Ricinguettio contiene l’identificativo del micropost B, altrimenti è nullo.

Inoltre, valgono i seguenti vincoli di integrità referenziale: IDUtente1 e IDUtente2 in SEGUACE referenziano UTENTE. IDUtente in CINGUETTIO referencia UTENTE. Ricinguettio in CINGUETTIO referencia CINGUETTIO.

Produrre un’espressione in SQL che esprima la domanda:

“Elencare gli utenti (ID, nome e città) che hanno iniziato a seguire un altro utente dopo averne ricinguettato almeno un micropost”

Soluzione 1

```
SELECT *
FROM utente
WHERE IdUtente in
    (SELECT S.IdUtente1
     FROM seguace S JOIN cinguettio C1 ON (S.IdUtente1=C1.IdUtente)
     JOIN cinguettio C2 ON (C1.IdUtente2=C2.IdUtente AND
                          C1.Ricinguettio=C2.IdCinguettio)
     WHERE ( C1.DataC <  S.DataS )
    );
```

Altre soluzioni sono possibili.

Domanda 1

Le seguenti relazioni definiscono una base dati "Partite" per gestire partite di calcio tra squadre di città italiane. Gli attributi sottolineati sono le chiavi principali delle relazioni, l'attributo con * può avere valore nullo. Significato degli attributi non autoesplicativi: "CodF" = Codice fiscale di arbitro e giocatore, "PuntiSC" e "PuntiSO" indicano rispettivamente i punti realizzati dalla squadra che gioca in casa e quelli realizzati dalla squadra ospite. Una squadra vince se realizza più punti dell'altra oppure le due squadre pareggiano.

ARBITRO(CodF, Nome, DataNascita, CittàNascita, NazioneNascita, Categoria)

GIOCATORE(CodF, Nome, DataNascita, CittàNascita, NazioneNascita, GiocaIn*)

SQUADRA(NomeSquadra, Categoria, Città)

PARTITA(SquadraCasa, SquadraOspite, DataPartita, PuntiSC, PuntiSO, Arbitro)

Vincoli di integrità referenziale: "GiocaIn" referencia SQUADRA, "SquadraCasa" e "SquadraOspite" referenziano entrambi SQUADRA, "Arbitro" referencia ARBITRO.

Domanda 2

Con riferimento alla base dati "Partite" specificare in SQL l'interrogazione:

"Elencare le squadre che non hanno mai giocato insieme ma hanno giocato con una terza stessa squadra almeno una volta e hanno perso entrambe contro tale terza squadra".

Soluzione 2

```
SELECT s1.NomeSquadra, s2.NomeSquadra
FROM squadra s1 JOIN squadra s2 ON (s1.NomeSquadra < s2.NomeSquadra)
WHERE NOT EXISTS
    (SELECT * FROM partita p1
     WHERE (p1.SquadraCasa=s1.NomeSquadra AND p1.SquadraOspite=s2.NomeSquadra)
           OR (p1.SquadraCasa=s2.NomeSquadra AND p1.SquadraOspite=s1.NomeSquadra))
AND EXISTS
    (SELECT * FROM squadra s3
     WHERE s3.NomeSquadra<>s1.NomeSquadra AND s3.NomeSquadra<>s2.NomeSquadra
     AND EXISTS
         (SELECT * FROM partita p2
          WHERE (p2.SquadraCasa=s3.NomeSquadra AND
                 p2.SquadraOspite=s1.NomeSquadra AND PuntiSC>PuntiSO) OR
                 (p2.SquadraCasa=s1.NomeSquadra AND
                 p2.SquadraOspite=s3.NomeSquadra AND PuntiSO>PuntiSC))
     AND EXISTS
         (SELECT * FROM partita p3
          WHERE (p3.SquadraCasa=s3.NomeSquadra AND
                 P3.SquadraOspite=s3.NomeSquadra AND PuntiSC>PuntiSO) OR
                 (p3.SquadraCasa=s3.NomeSquadra AND
                 P3.SquadraOspite=s3.NomeSquadra AND PuntiSO>PuntiSC))));
```

Esonero 10/06/2020

Le seguenti relazioni definiscono una base di dati “MustEat” per gestire le ordinazioni e le consegne di pasti a domicilio. Gli attributi sottolineati sono le chiavi primarie delle relazioni.

UTENTE(Codice, Nome, Email, Città, IndirizzoPreferito)

INDIRIZZO(Utente, Indirizzo, Note)

ORDINE(Utente, Ristorante, DataOrdine, Prezzo, IndirizzoConsegna)

RISTORANTE(Codice, Nome, Tipo, Indirizzo, Città)

Vincoli di integrità referenziale:

INDIRIZZO(Utente) referencia UTENTE(Codice),

UTENTE(Codice, IndirizzoPreferito) referencia INDIRIZZO(Utente, Indirizzo),

ORDINE(Ristorante) referencia RISTORANTE(Codice) e

ORDINE(Utente, IndirizzoConsegna) referencia INDIRIZZO(Utente, Indirizzo).

Significato degli attributi: *IndirizzoPreferito* è l'indirizzo di consegna preferito per un dato utente; *Tipo* può assumere i valori *pizzeria*, *paninoteca*, *cucina etnica* e *cucina vegetariana*; *Prezzo* è il prezzo totale di un ordine. I rimanenti attributi sono autoesplicativi. Gli attributi sono tutti NOT NULL.

Esprimere in SQL tutte e tre le seguenti interrogazioni indicando a quale query si sta rispondendo (A, B, C):

(A - Bassa complessità) Indicare la data e il prezzo degli ordini (effettuati presso pizzerie) consegnati presso un indirizzo diverso da quello preferito dagli utenti che li hanno effettuati.

(B - Media complessità) Indicare nome e città degli utenti che hanno ordinato pasti ognuno di almeno 50 euro da almeno due ristoranti di tipo diverso.

(C - Alta complessità) Trovare i ristoranti che hanno totalizzato più ricavi rispetto agli altri ristoranti del loro stesso tipo (ovvero, i ristoranti per cui la somma totale del prezzo degli ordini è maggiore di quella degli altri ristoranti del loro stesso tipo).

Soluzioni.

A.

```
select o.DataOrdine, o.Prezzo
from utente u join ordine o on (u.Codice=o.Utente) join ristorante r on (o.Ristorante=r.Codice)
where o.IndirizzoConsegna<>u.IndirizzoPreferito and r.Tipo='pizzeria';
```

B.

```
select u.Nome, u.Città
from utente u join ordine o on (u.Codice=o.Utente) join ristorante r on (r.Codice=o.Ristorante)
where o.Prezzo>= 50
group by u.Codice
having count(distinct r.Tipo)>=2;
```

C.

```
with RicaviRistorante as (
select r.Codice, r.Nome, r.Tipo, sum(o.Prezzo) TotRicavi
from ristorante r join ordine o on (r.Codice=o.Ristorante)
group by r.Codice )
select *
from RicaviRistorante r1
where r1.TotRicavi >= all ( select r2.TotRicavi
from RicaviRistorante r2
where r2.Tipo=r1.Tipo and r2.Codice<>r1.Codice );
```

2. Esonero SQL V2

Le seguenti relazioni definiscono una base di dati “MustEat” per gestire le ordinazioni e le consegne di pasti a domicilio. Gli attributi sottolineati sono le chiavi primarie delle relazioni.

UTENTE(Codice, Nome, Email, Città, IndirizzoPreferito)

INDIRIZZO(Utente, Indirizzo, Note)

ORDINE(Utente, Ristorante, DataOrdine, Prezzo, IndirizzoConsegna)

RISTORANTE(Codice, Nome, Tipo, Indirizzo, Città)

Vincoli di integrità referenziale:

INDIRIZZO(Utente) referencia UTENTE(Codice),

UTENTE(~~Codice~~, ~~IndirizzoPreferito~~) referencia INDIRIZZO(Utente, Indirizzo),

ORDINE(~~Ristorante~~) referencia RISTORANTE(Codice) e

ORDINE(Utente, IndirizzoConsegna) referencia INDIRIZZO(Utente, Indirizzo).

Significato degli attributi: *IndirizzoPreferito* è l’indirizzo di consegna preferito per un dato utente; *Tipo* può assumere i valori *pizzeria*, *paninoteca*, *cucina etnica* e *cucina vegetariana*; *Prezzo* è il prezzo totale di un ordine. I rimanenti attributi sono autoesplicativi. Gli attributi sono tutti NOT NULL.

Esprimere in SQL tutte e tre le seguenti interrogazioni indicando a quale query si sta rispondendo (A, B, C):

(A - Bassa complessità) Riportare, ordinate per ordine alfabetico inverso, le email degli utenti che hanno ordinato pasti in pizzerie o in paninoteche di Torino.

(B - Media complessità) Trovare i tipi dei ristoranti di Nichelino che hanno servito almeno due clienti di Torino.

(C - Alta complessità) Tra i ristoranti che hanno ogni ordine al di sotto di 50 euro, trovare il ristorante con il ricavo maggiore nella fascia di ricavi totali tra 80 mila euro e 100 mila euro.

A.

```
select email
from utente u join ordine o on u.codice=o.utente join ristorante r on o.ristorante=r.codice
where (tipo='pizzeria' or tipo='paninoteca') and r.città='Torino'
order by email desc;
```

B.

```
select distinct tipo
from ristorante r join ordine o1 on r.codice=o1.ristorante join ordine o2 on r.codice=o2.ristorante join
utente u1 join o1.utente=u1.codice join utente u2 on o2.utente=u2.codice
where r.città='Nichelino' and u1.città='Torino' and u2.città='Torino' and u1.codice<>u2.codice;
```

C.

```
with risticavi as (
select ristorante, sum(prezzo) as totale
from ordine o
where ristorante not in (
select ristorante
from ordine
where prezzo ≥ 50)
group by ristorante
having sum(prezzo) ≥ 800000 and sum(prezzo) ≤ 100000)
select ristorante
from risticavi
where totale ≥ all (
select totale
from risticavi);
```

1. Esonero SQL V1

Le seguenti relazioni definiscono una base di dati “AllYouCanFit” per gestire esercizi della palestra. Gli attributi sottolineati sono le chiavi primarie delle relazioni.

UTENTE(Nome, Cognome, Età, Sesso)

VOCESCHEDA(Nome, Cognome, Ordine, Esercizio, NumRipetizioni, Durata)

ESERCIZIO(Nome, Tipo, Attrezzo)

Vincoli di integrità referenziale:

VOCESCHEDA(Nome, Cognome) referencia UTENTE(Nome, Cognome),

VOCESCHEDA(Esercizio) referencia ESERCIZIO(Nome).

Significato delle relazioni e degli attributi: *VOCESCHEDA* contiene le voci della scheda dell'utente con i vari esercizi, *Ordine* indica l'ordine in cui l'esercizio deve essere eseguito (1, 2, 3, ...) nella scheda, *Tipo* può assumere i valori *aerobico*, *posturale*, *tonicità*; *Attrezzo* può assumere i valori *cyclette*, *manubrio*, *elastico* ecc. e può essere NULL. Le rimanenti relazioni e attributi sono autoesplicativi.

Esprimere in SQL tutte e tre le seguenti interrogazioni indicando a quale query si sta rispondendo (A, B, C):

(A - Bassa complessità) Elencare, senza duplicati, i tipi di esercizi fatti da donne che hanno tra 20 e 30 anni.

(B - Media complessità) Trovare gli utenti e la durata media degli esercizi aerobici nella loro scheda per gli utenti la cui durata complessiva degli esercizi aerobici è superiore a 60 minuti.

(C - Alta complessità) Trovare l'utente che, nell'ultimo esercizio senza attrezzi della propria scheda, ha fatto più ripetizioni.

Soluzioni.

A.

```
select distinct e.tipo
from utente u join vocescheda s on u.nome=s.nome and u.cognome=s.cognome
join esercizio e on s.esercizio=e.nome
where sesso='femmina' and età>=20 and età<=30;
```

B.

```
select s1.nome, s1.cognome, avg(s1.durata)
from vocescheda s1 join esercizio e on s1.esercizio=e.nome
where s1.tipo='aerobico'
group by s1.nome, s1.cognome
having sum(s1.durata)>60;
```

C.

```
with UltimoEsercizio as ( select nome,cognome,numripetizioni
from vocescheda
where (nome,cognome,ordine)=(
select nome,cognome,max(ordine)
from vocescheda s join esercizio e on s.esercizio=e.nome
where attrezzo is null
group by nome,cognome))
select nome, cognome from UltimoEsercizio where numripetizioni=( select max(numripetizioni) from
UltimoEsercizio);
```

Le seguenti relazioni definiscono una base di dati “AllYouCanFit” per gestire esercizi della palestra. Gli attributi sottolineati sono le chiavi primarie delle relazioni.

UTENTE(Nome, Cognome, Età, Sesso)

VOCESCHEDA(Nome, Cognome, Ordine, Esercizio, NumRipetizioni, Durata)

ESERCIZIO(Nome, Tipo, Attrezzo)

Vincoli di integrità referenziale:

VOCESCHEDA(Nome, Cognome) referenzia UTENTE(Nome, Cognome), VOCESCHEDA(Esercizio) referenzia ESERCIZIO(Nome).

Significato delle relazioni e degli attributi: *VOCESCHEDA* contiene le voci della scheda dell’utente con i vari esercizi, *Ordine* indica l’ordine in cui l’esercizio deve essere eseguito (1, 2, 3, ...) nella scheda, *Tipo* può assumere i valori *aerobico*, *posturale*, *tonicità*; *Attrezzo* può assumere i valori *cyclette*, *manubrio*, *elastico* ecc. e può essere NULL. Le rimanenti relazioni e attributi sono autoesplicativi.

Esprimere in SQL tutte e tre le seguenti interrogazioni indicando a quale query si sta rispondendo (A, B, C):

(A - Bassa complessità) Elencare, in ordine alfabetico decrescente e senza duplicati, gli attrezzi usati da uomini nei primi tre esercizi delle loro schede.

(B - Media complessità) Considerando gli attrezzi che, negli esercizi di tipo posturale, non hanno più di 20 ripetizioni, mostrare il nome dell’attrezzo e la durata media degli esercizi.

(C - Alta complessità) Trovare l’attrezzo usato più a lungo (cioè, la cui somma delle durate di utilizzo è massima) negli esercizi di tipo aerobico presenti in schede in cui era l’unico attrezzo usato.

A.

```
select distinct e.attrezzo
from utente u join vocescheda s on u.nome=s.nome and u.cognome=s.cognome
join esercizio e on s.esercizio=e.nome
where e.attrezzo is not null and sesso='maschio' and ordine<=3
order by e.attrezzo desc;
```

B.

```
select e.attrezzo, avg(e.durata)
from vocescheda s join e esercizio on s.esercizio=e.nome
where e.tipo='posturale' and e.attrezzo is not null
group by e.attrezzo
having max(s.numripetizioni)<=20;
```

C.

```
with UnicoAttrezzoAerobico as (
select e.attrezzo, sum(s.durata) as SommaDurata
from vocescheda s join esercizio e on s.esercizio=e.nome
where e.attrezzo is not null and e.tipo='aerobico'
and not exists (
select *
from vocescheda s1 join esercizio e1 on s1.esercizio=e1.nome
where s1.nome=s.nome and s1.cognome=s.cognome and e1.attrezzo<>e.attrezzo)
group by e.attrezzo)
select attrezzo
from UnicoAttrezzoAerobico
where SommaDurata=(
select max(SommaDurata)
from UnicoAttrezzoAerobico);
```


Le seguenti relazioni definiscono una base di dati “MustEat” per gestire le ordinazioni e le consegne di pasti a domicilio. Gli attributi sottolineati sono le chiavi primarie delle relazioni.

UTENTE(Codice, Nome, Email, Città, IndirizzoPreferito)

INDIRIZZO(Utente, Indirizzo, Note)

ORDINE(Utente, Ristorante, DataOrdine, Prezzo, IndirizzoConsegna)

RISTORANTE(Codice, Nome, Tipo, Indirizzo, Città)

Vincoli di integrità referenziale:

INDIRIZZO(Utente) ~~referenzia~~ UTENTE(Codice),

UTENTE(Codice, IndirizzoPreferito) referenzia INDIRIZZO(Utente, Indirizzo),

ORDINE(Ristorante) referenzia RISTORANTE(Codice) e

ORDINE(Utente, IndirizzoConsegna) referenzia INDIRIZZO(Utente, Indirizzo).

Significato degli attributi: *IndirizzoPreferito* è l’indirizzo di consegna preferito per un dato utente; *Tipo* può assumere i valori *pizzeria*, *paninoteca*, *cucina etnica* e *cucina vegetariana*; *Prezzo* è il prezzo totale di un ordine. I rimanenti attributi sono autoesplicativi. Gli attributi sono tutti NOT NULL.

Esprimere in SQL tutte e tre le seguenti interrogazioni indicando a quale query si sta rispondendo (A, B, C):

(A - Bassa complessità) Riportare, ordinate per ordine alfabetico, le città da cui sono partiti ordini per ristoranti di cucina etnica o cucina vegetariana di Torino.

(B - Media complessità) Indicare, per ogni utente, la massima spesa per un singolo ordine per ciascun tipo di ristorante.

(C - Alta complessità) Trovare le pizzerie che hanno consegnato solo a utenti della propria città e i cui ordini sono in media non superiori a 30 euro.

Soluzioni.

A.

```
select distinct u.città CittaUtente
from utente u join ordine o on u.codice=o.utente join ristorante r on o.ristorante=r.codice
where (tipo='cucina etnica' or tipo='cucina vegetariana') and r.città='Torino'
order by CittaUtente;
```

B.

```
select Utente, Tipo, max(Prezzo) as MaxPrezzo
from ORDINE JOIN RISTORANTE On ORDINE.Ristorante = RISTORANTE.Codice
group by Utente, Tipo;
```

C.

```
select Ristorante, avg(Prezzo) AvgPrezzo
from ORDINE join RISTORANTE on ORDINE.Ristorante = RISTORANTE.Codice
where Ristorante
not in (
select RISTORANTE.codice
FROM RISTORANTE join ORDINE on RISTORANTE.Codice=ORDINE.Ristorante join UTENTE
on ORDINE.Utente = UTENTE.Codice
where RISTORANTE.Città <> UTENTE.Città
)
and RISTORANTE.Tipo = 'pizzeria'
group by Ristorante
having AvgPrezzo <=30;
```

Le seguenti relazioni definiscono una base di dati “MustEat” per gestire le ordinazioni e le consegne di pasti a domicilio. Gli attributi sottolineati sono le chiavi primarie delle relazioni.

UTENTE(Codice, Nome, Email, Città, IndirizzoPreferito)

INDIRIZZO(Utente, Indirizzo, Note)

ORDINE(Utente, Ristorante, DataOrdine, Prezzo, IndirizzoConsegna)

RISTORANTE(Codice, Nome, Tipo, Indirizzo, Città)

Vincoli di integrità referenziale:

INDIRIZZO(Utente) referencia UTENTE(Codice),

UTENTE(Codice, IndirizzoPreferito) referencia INDIRIZZO(Utente, Indirizzo),

ORDINE(Ristorante) referencia RISTORANTE(Codice) e

ORDINE(Utente, IndirizzoConsegna) referencia INDIRIZZO(Utente, Indirizzo).

Significato degli attributi: *IndirizzoPreferito* è l’indirizzo di consegna preferito per un dato utente; *Tipo* può assumere i valori *pizzeria*, *paninoteca*, *cucina etnica* e *cucina vegetariana*; *Prezzo* è il prezzo totale di un ordine. I rimanenti attributi sono autoesplicativi. Gli attributi sono tutti NOT NULL.

Esprimere in SQL tutte e tre le seguenti interrogazioni indicando a quale query si sta rispondendo (A, B, C):

(A - Bassa complessità) Riportare, ordinate per ordine alfabetico inverso, le città da cui sono partiti ordini per pizzerie situate in una città diversa.

(B - Media complessità) Indicare per ogni città da cui siano partiti ordini, la spesa media per ciascun tipo di ristorante, escludendo le paninoteche.

(C - Alta complessità) Tra gli utenti che hanno ordinato da almeno due ristoranti diversi dello stesso tipo, trovare l’utente che ha speso di più in un singolo ordine.

A.

```
select distinct u.città CittaUtente
from utente u join ordine o on u.codice=o.utente join ristorante r on o.ristorante=r.codice
where tipo='pizzeria' and r.città <>u.città
order by CittaUtente desc;
```

B.

```
select u.città, Tipo, avg(Prezzo) as PrezzoMedio
from UTENTE u join ORDINE o on u.codice=o.utente JOIN RISTORANTE r On o.Ristorante =
r.Codice
where r.Tipo <> 'paninoteca'
group by u.città, Tipo;
```

C.

```
with UtentiDueRist as (
select *
from UTENTE u
join ORDINE o1 on o1.Utente = u.Codice
join RISTORANTE r1 on o1.Ristorante = r1.Codice
join ORDINE o2 on o2.Utente = u.Codice
join RISTORANTE r2 on o2.Ristorante = r2.Codice
where r1.Codice > r2.Codice
and r1.Tipo = r2.Tipo)
select max(Prezzo) MaxPrezzo, o.Utente
from ORDINE o
where o.Utente in (select u.Codice from UtentiDueRist)
group by ORDINE.Utente
having MaxPrezzo >= ALL (
select Prezzo from ORDINE
where o.Utente in (select u.Codice from UtentiDueRist));
```

Le seguenti relazioni definiscono una base di dati “AllYouCanFit” per gestire esercizi della palestra. Gli attributi sottolineati sono le chiavi primarie delle relazioni.

UTENTE(Nome, Cognome, Eta, Sesso)

VOCESCHEDA(Nome, Cognome, Ordine, Esercizio, NumRipetizioni, Durata)

ESERCIZIO(Nome, Tipo, Attrezzo)

Vincoli di integrità referenziale:

VOCESCHEDA(Nome, Cognome) referenzia UTENTE(Nome, Cognome),

VOCESCHEDA(Esercizio) referenzia ESERCIZIO(Nome).

Significato delle relazioni e degli attributi: *VOCESCHEDA* contiene le voci della scheda dell'utente con i vari esercizi, *Ordine* indica l'ordine in cui l'esercizio deve essere eseguito (1, 2, 3, ...) nella scheda, *Tipo* può assumere i valori *aerobico*, *posturale*, *tonicità*; *Attrezzo* può assumere i valori *cyclette*, *manubrio*, *elastico* ecc. e può essere NULL. Le rimanenti relazioni e attributi sono autoesplicativi.

Esprimere in SQL tutte e tre le seguenti interrogazioni indicando a quale query si sta rispondendo (A, B, C):

(A - Bassa complessità) Elencare, senza duplicati, i tipi di esercizi fatti da maschi che non hanno nessuna vocale nel cognome.

(B - Media complessità) Tra gli utenti che hanno tra 50 e 60 anni e hanno fatto in media tra 20 e 30 ripetizioni, elencare gli utenti e la durata massima dei loro esercizi. N.B.: non usare sottoquery.

(C - Alta complessità) Tra gli utenti che hanno fatto solo esercizi con più di 30 ripetizioni (tranne che con la cyclette, per cui il numero di ripetizioni può essere qualunque), trovare l'utente più vecchio e il numero totale delle ripetizioni dei suoi esercizi. Si presti attenzione al fatto che non tutti gli esercizi usano attrezzi e che possono esserci più utenti con la stessa età.

Soluzioni.

A.

```
select distinct e.tipo
from utente u join vocescheda v on u.nome=v.nome and u.cognome=v.cognome
join esercizio e on v.esercizio=e.nome
where sesso='maschio' and not(u.cognome like '%a%' or u.cognome like '%A%' or u.cognome like '%e%' or u.cognome like '%E%' or u.cognome like '%i%' or u.cognome like '%I%' or u.cognome like '%o%' or u.cognome like '%O%' or u.cognome like '%u%' or u.cognome like '%U%');
```

B.

```
select v.nome, v.cognome, max(v.durata)
from utente u join vocescheda v on u.nome=v.nome and u.cognome=v.cognome
where u.eta>=50 and u.eta<=60
group by v.nome, v.cognome
having avg(v.numripetizioni)>=20 and avg(v.numripetizioni)<=30;
```

C.

```
with utenti30rip as ( select nome,cognome,eta,numripetizioni
from utente u join vocescheda v1 on u.nome=v1.nome and u.cognome=v.cognome
where not exists (
select *
from vocescheda v2 join esercizio e on v2.esercizio=e.nome
where v2.nome=u.nome and v2.cognome=u.cognome and
(attrezzo<>'cyclette' or attrezzo is null) and numripetizioni<=30))
select nome, cognome, sum(numripetizioni)
from utenti30rip
where eta=(
select max(eta)
from utenti30rip)
group by nome, cognome;
```

Le seguenti relazioni definiscono una base di dati “AllYouCanFit” per gestire esercizi della palestra. Gli attributi sottolineati sono le chiavi primarie delle relazioni.

UTENTE(Nome, Cognome, Eta, Sesso)

VOCESCHEDA(Nome, Cognome, Ordine, Esercizio, NumRipetizioni, Durata)

ESERCIZIO(Nome, Tipo, Attrezzo)

Vincoli di integrità referenziale:

VOCESCHEDA(Nome, Cognome) referencia UTENTE(Nome, Cognome),

VOCESCHEDA(Esercizio) referencia ESERCIZIO(Nome).

Significato delle relazioni e degli attributi: *VOCESCHEDA* contiene le voci della scheda dell'utente con i vari esercizi, *Ordine* indica l'ordine in cui l'esercizio deve essere eseguito (1, 2, 3, ...) nella scheda, *Tipo* può assumere i valori ~~aerobico, posturale, tonicità~~; *Attrezzo* può assumere i valori *cyclette, manubrio, elastico* ecc. e può essere NULL. Le rimanenti relazioni e attributi sono autoesplicativi.

Esprimere in SQL tutte e tre le seguenti interrogazioni indicando a quale query si sta rispondendo (A, B, C):

(A - Bassa complessità) Elencare, senza duplicati, gli attrezzi utilizzati da femmine il cui cognome inizia per “Azz” e finisce per “ina” ma non contiene “oli”.

(B - Media complessità) Tra gli utenti che hanno meno di 30 anni o più di 50 anni e hanno fatto esercizi con durata media compresa tra 5 e 10 minuti, elencare utenti e il minimo numero di ripetizioni tra i loro esercizi. N.B.: non usare sottoquery.

(C - Alta complessità) Tra gli utenti che hanno fatto solo esercizi della durata di meno di 7 minuti (tranne che con il TRX, per cui la durata può essere qualunque), trovare l'utente più giovane e la durata media dei suoi esercizi. Si presti attenzione al fatto che non tutti gli esercizi usano attrezzi e che possono esserci più utenti con la stessa età.

Soluzioni.

A.

```
select distinct e.attrezzo
from utente u join vocescheda v on (u.nome=v.nome and u.cognome=v.cognome)
join esercizio e on v.esercizio=e.nome
where sesso='femmina' and cognome like 'Azz%ina' and not(cognome like '%oli%');
```

B.

```
select v.nome, v.cognome, min(v.numripetizioni)
from utente u join vocescheda v on u.nome=v.nome and u.cognome=v.cognome
where u.eta<30 or u.eta>50
group by v.nome, v.cognome
having avg(v.durata)>=5 and avg(v.durata)<=10;
```

C.

```
with utenti7min as ( select nome,cognome,eta,durata
from utente u join vocescheda v1 on u.nome=v1.nome and u.cognome=v.cognome
where not exists (
select *
from vocescheda v2 join esercizio e on v2.esercizio=e.nome
where v2.nome=u.nome and v2.cognome=u.cognome and
(attrezzo<>'TRX' or attrezzo is null) and durata>=7))
select nome, cognome, avg(durata)
from utenti7min
where eta=(
select min(eta)
from utenti7min)
group by nome, cognome;
```

Le seguenti relazioni definiscono una base di dati “BloodyMary” per gestire un laboratorio di analisi del sangue. Gli attributi sottolineati sono le chiavi primarie delle relazioni.

UTENTE(Nome, Cognome, Eta, Sesso)

REFERTO(Nome, Cognome, Data, Analisi, Esito)

ANALISI(Nome, Tipo, Digiuno, Prezzo, NormaMin, NormaMax)

Vincoli di integrità referenziale:

REFERTO(Nome, Cognome) referenzia UTENTE(Nome, Cognome),

REFERTO(Analisi) referenzia ANALISI(Nome).

Significato delle relazioni e degli attributi. *REFERTO* contiene i singoli esiti di un referto per le varie analisi effettuate in una certa data, *Data* è la data di effettuazione dell’analisi ed è rappresentata come ‘YYYY/MM/DD’, *Esito* è un valore numerico che indica il risultato dell’analisi. *ANALISI* contiene le possibili analisi effettuate nel laboratorio: *Tipo* può assumere i valori *ematologia*, *endocrinologia*, *biochimica*, *coagulazione*; *Digiuno* indica se è richiesto il digiuno; *NormaMin* e *NormaMax* indicano l’intervallo dei valori nella norma. *Eta* può essere NULL quando non si conosce la data dell’utente. Le rimanenti relazioni e attributi sono autoesplicativi.

Esprimere in SQL tutte e tre le seguenti interrogazioni indicando a quale query si sta rispondendo (A, B, C):

(A - Bassa complessità) Elencare, senza duplicati e ordinati per cognome e nome, gli utenti con meno di 40 anni che hanno analisi fuori dalla norma.

(B - Media complessità) Trovare, tra le analisi che costano meno di 50 euro, quelle che nel 2020 hanno generato oltre 100 mila euro di ricavi. Mostrare il nome dell’analisi, il ricavo totale e la data in cui è stata effettuata per l’ultima volta.

(C - Alta complessità) Trovare gli utenti che hanno fatto tutte le analisi possibili.

Soluzioni.

A.

```
select distinct u.cognome, u.nome
from utente u join referto r on u.nome=r.nome and u.cognome=r.cognome
join analisi a on r.analisi=a.nome
where eta < 40 and (esito < normamin or esito > normamax)
order by u.cognome, u.nome;
```

B.

```
select distinct a.nome, sum(prezzo), max(data)
from utente u join referto r on u.nome=r.nome and u.cognome=r.cognome
join analisi a on r.analisi=a.nome
where data >= '2020/01/01' and data <= '2020/12/31' and prezzo < 50
group by a.nome
having sum(prezzo) > 100000;
```

C.

```
select distinct nome,cognome
from referto r
where not exists (
select *
from analisi a
where a.nome not in (
select r1.analisi
from referto r1
where r1.nome=r.nome and r1.cognome=r.cognome);
```

2. Esonero SQL V2

Le seguenti relazioni definiscono una base di dati “BloodyMary” per gestire un laboratorio di analisi del sangue. Gli attributi sottolineati sono le chiavi primarie delle relazioni.

UTENTE(Nome, Cognome, Eta, Sesso)

REFERTO(Nome, Cognome, Data, Analisi, Esito)

ANALISI(Nome, Tipo, Digiuno, Prezzo, NormaMin, NormaMax)

Vincoli di integrità referenziale:

REFERTO(Nome, Cognome) referencia UTENTE(Nome, Cognome),

REFERTO(Analisi) referencia ANALISI(Nome).

Significato delle relazioni e degli attributi. *REFERTO* contiene i singoli esiti di un referto per le varie analisi effettuate in una certa data, *Data* è la data di effettuazione dell’analisi ed è rappresentata come ‘YYYY/MM/DD’, *Esito* è un valore numerico che indica il risultato dell’analisi. *ANALISI* contiene le possibili analisi effettuate nel laboratorio: *Tipo* può assumere i valori *ematologia*, *endocrinologia*, *biochimica*, *coagulazione*; *Digiuno* indica se è richiesto il digiuno; *NormaMin* e *NormaMax* indicano l’intervallo dei valori nella norma. *Eta* può essere NULL quando non si conosce la data dell’utente. Le rimanenti relazioni e attributi sono autoesplicativi.

Esprimere in SQL tutte e tre le seguenti interrogazioni indicando a quale query si sta rispondendo (A, B, C):

(A - Bassa complessità) Elencare, senza duplicati e ordinate per cognome e nome decrescenti, le donne con più di 60 anni che hanno analisi nella norma.

(B - Media complessità) Trovare, considerando solo le analisi che costano più di 20 euro, i tipi di analisi che nel 2019 sono stati eseguiti meno di 100 volte. Mostrare il tipo dell’analisi, il numero totale e il prezzo medio.

(C - Alta complessità) Trovare, senza utilizzare operatori aggregati, gli utenti che hanno fatto tutte le analisi per cui non è richiesto il digiuno.

Soluzioni.

A.

```
select distinct u.cognome, u.nome
from utente u join referto r on u.nome=r.nome and u.cognome=r.cognome
join analisi a on r.analisi=a.nome
where eta > 60 and sesso='F' and esito >= normamin and esito <= normamax
order by u.cognome desc, u.nome desc;
```

B.

```
select a.tipo, count(*), avg(prezzo)
from utente u join referto r on u.nome=r.nome and u.cognome=r.cognome
join analisi a on r.analisi=a.nome
where data >= '2019/01/01' and data <= '2019/12/31' and prezzo > 20
group by a.tipo
having count(*) < 100;
```

C.

```
select distinct nome,cognome
from referto r
where not exists (
select *
from analisi a
where a.digiuno=false and a.nome not in (
select r1.analisi
from referto r1
where r1.nome=r.nome and r1.cognome=r.cognome));
```

Le seguenti relazioni definiscono una base di dati “**Pizzeria**” per gestire gli ordini di una pizzeria. Gli attributi sottolineati sono le chiavi primarie delle relazioni.

ORDINE(Pizza, Tavolo, Data, Quantità, Note)

PIZZA(NomePizza, Prezzo, Tipo)

CONTIENE(Pizza, Ingrediente)

INGREDIENTE(NomeIngrediente, Vegetariano, SenzaGlutine)

Vincoli di integrità referenziale: ORDINE(Pizza) referencia PIZZA(NomePizza), CONTIENE(Pizza) referencia PIZZA(NomePizza), CONTIENE(Ingrediente) referencia INGREDIENTE(NomeIngrediente).

“Tipo” può assumere i valori "Normale" e “Gourmet”. “Vegetariano” e “SenzaGlutine” sono booleani.

I rimanenti attributi sono autoesplicativi. Gli attributi sono tutti NOT NULL.

Con riferimento alla base di dati "Pizzeria" esprimere in SQL le seguenti interrogazioni.

Domanda 1 (bassa complessità).

Trovare le date e i tavoli che hanno ordinato almeno due pizze con l'ananas o con le pere.

Soluzione 1.

```
SELECT DISTINCT Data, Tavolo
FROM ordine O JOIN pizza P ON (O.Pizza=P.NomePizza)
      JOIN contiene C ON (P.NomePizza=C.Pizza)
      JOIN ingrediente I ON (C.Ingrediente=I.NomeIngrediente)
WHERE O.Quantità>=2 AND (I.NomeIngrediente='Ananas' OR I.NomeIngrediente='Pera');
```

Domanda 2 (media complessità).

Mostrare le date in cui dei tavoli hanno speso più di 50 euro in pizze gourmet.

Soluzione 2.

```
SELECT DISTINCT O.Data
FROM ordine O JOIN pizza P ON (O.Pizza=P.NomePizza)
WHERE P.Tipo='Gourmet'
GROUP BY O.Data, O.Tavolo
HAVING SUM(P.Prezzo*O.Quantità)>50;
```

Domanda 3 (alta complessità).

Trovare il conto medio per l'anno 2018 degli ordini composti solo da pizze vegetariane (cioè pizze in cui tutti gli ingredienti sono vegetariani).

Soluzione 3.

```
WITH ContiVegetariani AS (
  SELECT O.Data, O.Tavolo, SUM(P.Prezzo*O.Quantità) Conto
  FROM ordine O JOIN pizza P ON (O.Pizza=P.NomePizza)
  WHERE O.Data, O.Tavolo NOT IN (
    SELECT O1.Data, O1.Tavolo
    FROM ordine O1 JOIN pizza P1 ON (O1.Pizza=P1.NomePizza)
      JOIN contiene C ON (P1.NomePizza=C.Pizza)
      JOIN ingrediente I ON (C.Ingrediente=I.NomeIngrediente)
    WHERE I.Vegetariano=False)
  GROUP BY O.Data, O.Tavolo)
SELECT AVG(C.Conto)
FROM ContiVegetariani C
WHERE C.Data>='01-01-2018' AND C.Data<'01-01-2019';
```


Le seguenti relazioni definiscono una base di dati “**PFWWrestling**” per gestire le informazioni di una federazione di wrestling. Gli attributi sottolineati sono le chiavi primarie delle relazioni.

WRESTLER(RingName, Nome, Cognome, Sesso, Peso, StatoProvenienza)

CINTURA(Campione, Titolo, NumeroDiSuccessi)

STABLE(NomeStable, Capitano*)

COMPOSIZIONESTABLE(NomeStable, NomeComponente)

Vincoli di integrità referenziale:

CINTURA(Campione) referencia WRESTLER(RingName),

STABLE(Capitano) referencia WRESTLER(RingName),

COMPOSIZIONESTABLE(NomeStable) referencia STABLE(NomeStable),

COMPOSIZIONESTABLE(NomeComponente) referencia WRESTLER(RingName),

STABLE(NomeStable, Capitano) referencia COMPOSIZIONESTABLE(NomeStable, NomeComponente).

“Sesso” può assumere i valori “Uomo” e “Donna”. Capitano può essere NULL. Titolo può assumere i valori “assoluto”, “intercontinentale” e “femminile”. Una stable è composta da almeno tre wrestler. Solo le donne possono avere un titolo femminile. I rimanenti attributi sono autoesplicativi.

Con riferimento alla base di dati "PFWWrestling" esprimere in SQL le seguenti interrogazioni.

Domanda 1 (bassa complessità).

Trovare gli stati di provenienza dei capitani di stable che hanno vinto almeno 3 volte il titolo assoluto.

```
SELECT DISTINCT StatoProvenienza
FROM wrestler JOIN stable ON (Capitano = RingName)
      JOIN cintura ON (Capitano = Campione)
WHERE Titolo = 'assoluto' AND NumeroDiSuccessi >= 3;
```

Domanda 2 (media complessità).

Trovare ringname, nome e cognome dei wrestlers canadesi che fanno parte di almeno 5 stable i cui capitani hanno vinto il titolo intercontinentale.

```
SELECT RingName, Nome, Cognome
FROM wrestler JOIN composizionestable c ON (RingName = NomeComponente)
      JOIN stable s ON (s.NomeStable = c.NomeStable)
      JOIN cintura ON (Capitano = Campione)
WHERE StatoProvenienza = 'Canada' AND Titolo = 'intercontinentale'
GROUP BY RingName
HAVING COUNT(DISTINCT NomeStable)>=5;
```

Domanda 3 (alta complessità).

Tra le stable composte da soli uomini, trovare quelle che hanno il minimo numero di componenti.

```
WITH StableSenzaDonne AS (
  SELECT s.NomeStable, COUNT(s.NomeComponente) NumeroComponenti
  FROM composizionestable s
  WHERE NOT EXISTS (
    SELECT *
    FROM composizionestable c
      JOIN wrestler w ON (w.RingName = c.NomeComponente)
    WHERE c.NomeStable=s.NomeStable AND w.Sesso = 'Donna')
  GROUP BY s.NomeStable)
SELECT NomeStable
FROM StableSenzaDonne
WHERE NumeroComponenti = (SELECT MIN(NumeroComponenti)
                          FROM StableSenzaDonne);
```

Le seguenti relazioni definiscono una base di dati “**NerdFlix**” per gestire una piattaforma di streaming online di serie TV. Gli attributi sottolineati sono le chiavi primarie delle relazioni.

SERIETV(NomeSerie, Produttore, Nazione, Genere)

EPISODIO(NomeSerie, Stagione, Episodio, TitoloEpisodio, Anno)

AUDIO(NomeSerie, Stagione, Episodio, Lingua, AdattatoreDialoghi*, Originale)

SOTTOTITOLI(NomeSerie, Stagione, Episodio, Lingua)

Vincoli di integrità referenziale:

EPISODIO(NomeSerie) referencia SERIETV(NomeSerie),

AUDIO(NomeSerie,Stagione,Episodio) referencia EPISODIO(NomeSerie,Stagione,Episodio),

SOTTOTITOLI(NomeSerie,Stagione,Episodio) referencia EPISODIO(NomeSerie,Stagione,Episodio).

“Genere” può assumere i valori “Animazione”, “Dramma”, “Fantascienza” e “Commedia”. “Originale” può assumere i valori True (se l’audio è quello della lingua originale) e False (se si tratta di un doppiaggio). I rimanenti attributi sono autoesplicativi. AdattatoreDialoghi può essere NULL.

A titolo di esempio, la tupla della relazione EPISODIO <'Stranger Things', 3, 2, 'Capitolo due – Incubi', 2019> si riferisce al secondo episodio della terza stagione della serie TV “Stranger Things”. **Non è detto che tutti gli episodi (o che tutte le stagioni) di una serie siano presenti nella base di dati.**

Con riferimento alla base di dati "NerdFlix" esprimere in SQL le seguenti interrogazioni.

Domanda 1 (bassa complessità).

Trovare le serie TV italiane di genere “Commedia” con almeno un episodio sottotitolato in francese.

```
SELECT DISTINCT s.NomeSerie
FROM serietv se JOIN sottotitoli so ON (se.NomeSerie=so.NomeSerie)
WHERE se.Genere='Commedia' AND se.Nazione='Italia' AND so.Lingua='Francese';
```

Domanda 2 (media complessità).

Mostrare le serie TV di cui sono stati prodotti almeno 20 episodi con audio originale in tedesco.

```
SELECT e.NomeSerie
FROM episodio e JOIN audio a ON (e.NomeSerie=a.NomeSerie
                                AND e.Stagione=a.Stagione AND e.Episodio=a.Episodio)
WHERE a.Lingua='Tedesco' AND a.Originale='true'
GROUP BY e.NomeSerie
HAVING COUNT(*)>=20;
```

Domanda 3 (alta complessità).

Tra le serie TV che **non** hanno il numero massimo di episodi, trovare quella (o quelle) con più stagioni prodotte (non utilizzare costrutti come LIMIT, ROWNUM, TOP).

```
WITH NumEpisodiSerieTV AS (
    SELECT e.NomeSerie, COUNT(*) NumEpisodi
    FROM episodio e
    GROUP BY e.NomeSerie),
NumStagioniSerieTVEscluseMaxNumEpisodi AS (
    SELECT e.NomeSerie, COUNT(DISTINCT e.NumStagione) NumStagioniSTV
    FROM episodio e
    WHERE e.NomeSerie IN (SELECT NomeSerie
                          FROM NumEpisodiSerieTV
                          WHERE NumEpisodi < ANY (SELECT NumEpisodi
                                                    FROM NumEpisodiSerieTV))
    GROUP BY e.NomeSerie)
SELECT NomeSerie
FROM NumStagioniSerieTVEscluseMaxNumEpisodi
WHERE NumStagioniSTV = (SELECT MAX(NumStagioniSTV)
                       FROM NumStagioniSerieTVEscluseMaxNumEpisodi);
```

Le seguenti relazioni definiscono una base di dati “**Titoli**” per gestire il palmarès delle squadre di calcio italiane. Gli attributi sottolineati sono le chiavi primarie delle relazioni.

SQUADRA(NomeSquadra, Città, AnnoFondazione)

TITOLO(NomeTitolo, Tipo, AnnoIstituzione)

CONQUISTA(NomeSquadra, NomeTitolo, AnnoConquista)

CONQUISTA(NomeSquadra) referencia SQUADRA(NomeSquadra),

CONQUISTA(NomeTitolo) referencia TITOLO(NomeTitolo),

“Tipo” può assumere i valori “Europeo”, “Nazionale”. Gli altri attributi sono autoesplicativi.

Con riferimento alla base di dati “Titoli” esprimere in SQL le seguenti interrogazioni.

Domanda 1 (bassa complessità).

Mostrare, per ogni città, il numero di squadre che hanno conquistato il titolo “Scudetto” nell’anno della loro fondazione.

Soluzione 1.

```
SELECT s.Città, COUNT(s.NomeSquadra)
FROM squadra s JOIN conquista c ON (s.NomeSquadra=c.NomeSquadra)
WHERE s.AnnoFondazione=c.AnnoConquista AND c.NomeTitolo='Scudetto'
GROUP BY s.Città;
```

Domanda 2 (media complessità).

Trovare la squadra o le squadre che, in un solo anno, hanno realizzato il “triple” ovvero hanno ottenuto nello stesso anno i titoli “Scudetto” e “Coppa Italia” e almeno un titolo tra “UEFA Champions League” e “Coppa dei Campioni”. Mostrare anche l’anno in cui il triple è stato realizzato.

Soluzione 2.

```
SELECT s.NomeSquadra, c1.AnnoConquista
FROM squadra s JOIN conquista c1 ON (s.NomeSquadra=c1.NomeSquadra)
                JOIN conquista c2 ON (s.NomeSquadra=c2.NomeSquadra AND
                                     c1.AnnoConquista=c2.AnnoConquista)
                JOIN conquista c3 ON (s.NomeSquadra=c3.NomeSquadra AND
                                     c3.AnnoConquista=c2.AnnoConquista)
WHERE c1.NomeTitolo='Scudetto' AND c2.NomeTitolo='Coppa Italia' AND
      (c3.NomeTitolo='Coppa dei Campioni' OR c3.NomeTitolo='UEFA Champions League');
```

Domanda 3 (alta complessità).

Tra le squadre che dal 2000 non hanno vinto nessun titolo Europeo, trovare quella che, nello stesso periodo, ha conquistato più scudetti.

Soluzione 3.

```
WITH SquadreSenzaTitoliEuropei AS (
  SELECT c.NomeSquadra as Squadra, COUNT(*) NumeroScudetti
  FROM conquista c
  WHERE AnnoConquista>=2000 AND c.NomeTitolo='Scudetto' AND NOT EXISTS
    (SELECT *
     FROM conquista c1 JOIN TITOLO t ON (c1.NomeTitolo=t.NomeTitolo)
     WHERE c1.NomeSquadra=c.NomeSquadra AND t.Tipo='Europeo' AND
           AnnoConquista>=2000)
  GROUP BY Squadra)
SELECT Squadra
FROM SquadreSenzaTitoliEuropei
WHERE NumeroScudetti = (SELECT MAX(NumeroScudetti) FROM SquadreSenzaTitoliEuropei);
```

Esercizi Slide

Elencare tutti i fornitori il cui nome contiene una *h*

```
select * from S where Sname like '%h%';
```

Elencare tutti i fornitori il cui nome contiene una *s* (minuscola o maiuscola)

```
select * from S where Sname like '%s%' or Sname like '%S%';
```

Elencare tutti i fornitori che hanno nel nome una *a* e terminano con *s*

```
select * from S where Sname like '%a%s';
```

Elencare tutti i fornitori il cui nome contiene sia una *a* che una *e*

```
select * from S where Sname like '%a%' and Sname like '%e%';
```

Elencare tutti i fornitori il cui nome contiene tutte le vocali

```
select * from S where Sname like '%a%' and Sname like '%e%' and Sname like '%i%' and Sname like '%o%' and Sname like '%u%';
```

Elencare tutti i fornitori il cui nome coincide con il nome di una parte

```
select * from S,P where Sname=PName;
```

Elencare tutte le parti con un nome lungo almeno 4 caratteri

```
select * from P where Pname like '____%'
```

Elencare il nome di tutte le parti rosse con peso compreso tra 13 kg e 17 kg

```
select Pname from P where Color='Red' and (Weight>=13 and Weight<=17);
```

Elencare il nome di tutti i produttori di Atene (Athens) con Status inferiore a 20

```
select Sname from S where City='Athens' AND Status < 20;
```

Elencare nome, colore e peso delle parti e nome del fornitore

```
select PName, Color, Weight, Sname from S, P where S.City=P.City;
```

Elencare, in ordine alfabetico crescente, il nome dei fornitori che hanno evaso almeno una volta ordini di almeno 300 bulloni (bolt) o di almeno 300 dadi (nut)

```
select distinct Sname from S, SP, P where S.SNum=SP.SNum and P.PNum=SP.Pnum and QTY>=300 and (PName='Bolt' or PName='Nut') order by SName;
```

Elencare, in ordine di status decrescente, i fornitori che hanno fornito parti con peso fuori dall'intervallo [14 kg, 17 kg]

```
select distinct SName, Status from S, SP, P where S.SNum=SP.SNum AND P.PNum=SP.Pnum and not(Weight>=14 and Weight<=17) order by Status desc;
```

Elencare tutti i fornitori con Status superiore a 20 e la quantità delle parti eventualmente fornite

```
select S.*, Qty from S left join SP on S.SNum=SP.SNum where Status>20;
```

Elencare i nomi di tutte le parti di colore verde e le città dei loro eventuali fornitori

```
select distinct PName, S.City from P left join SP on P.PNum=SP.PNum left join S on SP.SNum=S.SNum where Color='Green';
```

Elencare tutti i fornitori che hanno forniture minori di 200 parti (e quindi anche i fornitori che non hanno fornito nulla). Il risultato deve comprendere il nome del fornitore e la quantità delle parti eventualmente fornite.

```
select distinct SName, Qty from S left join SP on S.SNum=SP.SNum where Qty<200 or Qty is null;
```

Elencare tutte le coppie di parti disponibili nella stessa città ma di colore diverso (mostrare codice delle parti e nome della città)

```
select P1.PNum as Part1, P2.PNum as Part2, P2.City from P P1 join P P2 on P1.City=P2.City where P1.PNum<P2.PNum and P1.Color<>P2.Color;
```

Elencare tutte le coppie di parti fornite dallo stesso fornitore (mostrare nome del fornitore, codice e nome delle parti)

```
select SName, P1.PNum, P1.PName, P2.PNum, P2.Pname from SP SP1 join SP SP2 on SP1.SNum=SP2.SNum join P P1 on P1.PNum=SP1.PNum join P P2 on P2.PNum=SP2.PNum join S on S.SNum=SP1.SNum where SP1.PNum<SP2.PNum;
```

Estrarre la quantità totale di parti rosse fornite da ciascun fornitore (mostrare nome del fornitore e quantità totale di parti)

```
select SName as SupplierName, sum(QTY) as TotQTY from SP join S on S.SNum=SP.SNum join P on SP.PNum=P.PNum where Color='Red' group by SP.SNum, S.SName;
```

Estrarre la quantità totale di parti rosse fornite da ciascun fornitore compresi i fornitori che non forniscono nessuna parte , per i quali dovete mostrare 0 (mostrare nome del fornitore e quantità totale di parti). Nel risultato ci aspettiamo di avere Adams, che non fornisce nessuna parte, ma non Blake, che fornisce solo parti verdi.

```
select SName as SupplierName, coalesce(sum(QTY),0) as TotQTY from S left join SP on S.SNum=SP.SNum left join P on SP.PNum=P.PNum where Color='Red' or Color is null group by SP.SNum, S.SName;
```

Estrarre la quantità totale di parti rosse fornite da ciascun fornitore compresi i fornitori che non forniscono nessuna parte rossa (mostrare nome del fornitore e quantità totale di parti) (suggerimento: sfruttare la condizione del join). Nel risultato ci aspettiamo di avere sia Adams, che non fornisce nessuna parte, che Blake, che fornisce solo parti verdi.

```
select SName as SupplierName, coalesce(sum(QTY),0) as TotQTY from S left join (SP join P on SP.PNum=P.PNum) on S.SNum=SP.SNum and Color='Red' group by SP.SNum, S.SName;
```

Estrarre le città in cui ci sono almeno due fornitori che hanno fornito ognuno almeno due prodotti di diverso colore (suggerimento: scrivere prima la query che estrae le informazioni sulle coppie di parti di diverso colore fornite dallo stesso fornitore)

```
select S.City from S join SP SP1 on S.SNum=SP1.SNum join SP SP2 on SP1.SNum=SP2.SNum join P P1 on SP1.PNum=P1.PNum join P P2 on SP2.PNum=P2.PNum where P1.Color<P2.Color group by S.City having count(distinct S.SNum)>=2;
```

Elencare i fornitori che hanno fornito parti disponibili a Londra (con costruito in)

```
select distinct Snum from SP where PNum in ( select Pnum from P where City = 'London' );
```

Elencare i fornitori che hanno fornito parti disponibili a Londra (con costruito any)

```
select distinct SNum from SP where PNum = any ( select Pnum from P where City = 'London' );
```

Elencare le città in cui non vi sono fornitori con status minore della media (con costruito in/not in)

```
select distinct City from S where City not in ( select City from S where Status < ( select avg(Status) from S ) );
```

Elencare le città in cui non vi sono fornitori con status minore della media (con costruito all)

```
select distinct City from S where City <> all ( select City from S where Status < ( select avg(Status) from S ) );
```

Trovare i codici dei prodotti che hanno il peso massimo (come esercizio sulle query correlate, scrivere una versione determinando il peso massimo come il peso non inferiore ai pesi di tutti gli altri prodotti e un'altra versione con not exists)

Soluzione con costruito all

```
select P1.Pnum from P P1 where P1.Weight >= all ( select P2.Weight from P P2 where P1.PNum <> P2.PNum );
```

Soluzione con costruito not exists

```
select P1.PNum from P P1 where not exists ( select * from P P2 where P2.Weight > P1.Weight );
```

Trovare i nomi dei fornitori che forniscono tutte le parti (senza utilizzare operatori aggregati)

(suggerimento: scrivere prima una query che trovi le parti non fornite da uno specifico fornitore, ad es. S2)

```
select Sname from S where not exists ( select * from P where P.PNum not in ( select SP.Pnum from SP where SP.SNum = S.SNum ) );
```

Trovare i nomi dei fornitori che forniscono almeno tutti i prodotti forniti da 'S2' (senza utilizzare

operatori aggregati) (suggerimento: scrivere prima una query che trovi i prodotti forniti da S2 ma non da uno specifico fornitore, ad esempio S3)

```
select Sname from S where not exists ( select * from SP SP1 where SP1.SNum = 'S2' and SP1.PNum not in ( select SP2.Pnum from SP SP2 where SP2.SNum = S.SNum ) );
```